

Programming notes for Assignment 1

The Python, NumPy, SciPy and scikit-learn the notebooks use

Fall 2026

What this is

Assignment 1 hands you four notebooks. Each has *provided* cells, which you run as they are, and *TODO* cells, which you complete. Most *TODO* cells suggest useful functions; this reference explains those calls: what each one takes, what it returns, and what its error messages mean. The companion handout, *Vectors, distances, and dissimilarity matrices*, covers the mathematics of vectors, dot products, and dissimilarities.

These notes explain the notation and library calls used in Assignment 1. Follow each notebook's instructions about which calculations you must implement yourself. Use the small examples to understand each operation before applying it in your notebook.

Every example below assumes these two imports, which the notebooks' setup cells run for you:

```
import numpy as np
import matplotlib.pyplot as plt
```

Examples marked *usage pattern* show a call's arguments and result without a runnable input; the others run as printed.

Check shapes first. Start by comparing the actual shape of a result with the shape you expected. Matching shapes are necessary but not sufficient: the values and the order of the items also need checking.

1 Python essentials

1.1 Containers

```
xs = [3, 1, 4]           # list: grows, holds anything
xs.append(1)             # xs is now [3, 1, 4, 1]
d = {'cat': 0, 'dog': 1} # dict: look things up by name
d['cat']                  # -> 0
shape = (120, 4096)       # tuple: fixed; shapes are tuples
shape[0]                  # -> 120
```

A NumPy array is the fourth container and the one that matters: a block of numbers of one type with a shape. Lists are for bookkeeping; arrays are for arithmetic.

1.2 Return a result so later code can use it

```
def halfway(a, b):
    """The point midway between a and b."""
    return (a + b) / 2

m = halfway(np.array([0., 2.]), np.array([4., 4.])) # array([2., 3.] )
```

Printing displays a value; returning passes it to the caller. A function with no return statement returns `None`. The TODO cells that ask for a function will call it later in the notebook, so it must return its result.

1.3 Loops and comprehensions

```
squares = [v ** 2 for v in xs] # a loop that builds a list: [9, 1, 16, 1]
for i, value in enumerate(xs): # enumerate: index and value together
    print(i, value)           # prints 0 3, then 1 1, then 2 4, then 3 1
```

Loops over layers, models, or seeds are the normal way to organise an analysis. NumPy applies arithmetic to whole arrays at once (§4), so a loop over the entries of an array is rarely needed.

1.4 Reading an error message

Read the *last* line first: it names the error. Then read upward to find the line of *your* code (not the library's) that caused it. §8 lists the ones you will meet.

2 Reading printed shapes

Printed	Read it as
(120, 4096)	a matrix: 120 stimuli, each a vector of 4,096 features
(4096,)	a one-dimensional array of 4,096 numbers (the shape is a one-element tuple)
(120, 1)	a column: 120 rows of one number each; broadcasts against (120, 4096)
(120,)	120 numbers; does not broadcast against (120, 4096)
(120, 120)	a square matrix over 120 stimuli: a similarity or dissimilarity matrix
(7140,)	the upper triangle of a 120×120 matrix: $120 \cdot 119/2$ pairs
(120, 2)	120 points in the plane: an MDS, PCA or t-SNE embedding
(120, 224, 224, 3)	120 colour photographs, 224×224 pixels, 3 channels (Parts 1–2)
(120, 150528)	the same photographs, each flattened to one vector ($224 \cdot 224 \cdot 3$)
(1224, 200, 200)	1,224 greyscale images of 200×200 pixels (Part 3)
(1224, 40000)	the same images, each flattened to one vector ($200 \cdot 200$)
()	a single number wrapped in an array; <code>x.item()</code> extracts it

Write the shape you expect in a comment before you run the line, then compare. The notebooks' setup cells print shapes at every load for exactly this reason.

3 A debugging recipe

1. **Shape.** Print it. Is it what you expected, and the right way round?
2. **Type and range.** `X.dtype`, `X.min()`, `X.max()`. Unsigned-integer arithmetic (`uint8`) wraps around; lower-precision floating point (`float32`) accumulates roundoff.
3. **Missing values.** `np.isnan(X).sum()`.
4. **A case you can check by hand.** Run your code on a small array such as `A` in §4.1 and compare with a calculator.
5. **Say what the line computes**, in one sentence, before fixing it. If you cannot, work through a smaller example, reread the companion handout, or ask.

4 NumPy: arrays, shapes, and the axis argument

4.1 Shape, dtype, and one row per stimulus

```
A = np.array([[1., 2.],
              [3., 6.],
              [5., 4.]])
A.shape      # (3, 2): 3 rows, 2 columns
A.ndim       # 2
A.dtype      # float64
```

The convention. A data matrix is **stimuli by features**: one row per image, one column per pixel, unit, or neuron. So `X[i]` is the vector for stimulus *i* and `X.shape[0]` is how many stimuli there are. The assignment's matrices are, for example, (120, 4096): 120 images, 4,096 units.

4.2 Arithmetic on arrays: * is not @

```
A * A      # ELEMENTWISE product, same shape as A: [[1, 4], [9, 36], [25, 16]]
A ** 2     # elementwise square, the same numbers; ** is the power operator
A @ A.T    # MATRIX product: (3, 2) @ (2, 3) -> (3, 3), row dotted with row
3 ^ 2      # 1, not 9: ^ is bitwise XOR in Python, never exponentiation
v = np.array([1., 2., 3.])
v.T.shape  # (3,): transposing a 1-D array does nothing
v[:, None].shape # (3, 1): this is how you make a column
[1, 2] + [3, 4], [1, 2] * 2 # on LISTS: [1, 2, 3, 4] and [1, 2, 1, 2]
```

`+`, `-`, `*`, `/` and `**` act entry by entry on arrays; `@` (or `np.dot`) is the matrix product of the companion handout. On lists, `+` concatenates and `*` repeats. Convert with `np.array`.

4.3 Indexing and slicing

```
A[0]          # array([1., 2.])    row 0
A[:, 1]       # array([2., 6., 4.]) column 1
A[:2]        # rows 0 and 1
A[-1]        # array([5., 4.])    last row
A[1, 0]       # 3.0               row 1, column 0
```

Indices start at 0, so the last valid row of a 120-row matrix is 119. Basic slices such as `A[:2]` share memory with `A`: changing one changes the other. Use `.copy()` for an independent array. Boolean and integer-array indexing (§4.6, §4.9) return copies.

4.4 Reshaping

```
np.arange(6)          # array([0, 1, 2, 3, 4, 5])
np.arange(6).reshape(2, 3) # [[0, 1, 2], [3, 4, 5]]: same numbers, read row by row
np.arange(6).reshape(2, -1) # -1 means "whatever fits": also (2, 3)
imgs.reshape(len(imgs), -1) # usage pattern: (n, 200, 200) -> (n, 40000)
np.stack([A[0], A[1]])     # two vectors -> one (2, 2) matrix, one per row
```

4.5 The axis argument

Reductions and axis. For a reduction such as `mean`, `sum` or `std`, `axis` names the dimension being combined. By default that dimension disappears from the result; `keepdims=True` keeps it with length 1. `A.mean(axis=0)` combines the rows and leaves one number per column; `A.mean(axis=1)` combines the columns and leaves one number per row.

```
A.mean(axis=0)          # array([3., 4.])      shape (2,)
A.mean(axis=1)          # array([1.5, 4.5, 4.5]) shape (3,)
A.mean(axis=1, keepdims=True) # array([[1.5], [4.5], [4.5]]) shape (3, 1)
A.sum(), A.max(axis=0), A.std(axis=1) # same rule for every reduction
np.cumsum([2, 3, 1])     # running total: array([2, 5, 6])
```

4.6 Broadcasting

When two arrays have different shapes, NumPy stretches any size-1 dimension to match. Compare shapes *from the right*: each pair must be equal, or one of them must be 1. A missing leading dimension counts as 1.

```

A + np.array([10., 20.])          # (3,2) + (2,) -> one offset per column
# array([[11., 22.],
#        [13., 26.],
#        [15., 24.]])
A + np.array([[10.], [20.], [30.]]) # (3,2) + (3,1) -> one offset per row
# array([[11., 12.],
#        [23., 26.],
#        [35., 34.]])
A + np.array([10., 20., 30.])      # (3,2) + (3,) -> expected error:
# ValueError: operands could not be broadcast together with shapes (3,2) (3,)

```

axis selects which entries a reduction combines; keepdims (or `v[:, None]`) controls the result's shape so that it broadcasts back against the matrix the way you intend. The third line fails because `(3,)` lines up against the *columns* (2), not the rows. A wrong broadcast can also run without any error when the sizes happen to match, so check the result's shape and a value or two.

4.7 Boolean masks

```

labels = np.array(['a', 'b', 'a'])
mask = (labels == 'a')          # array([ True, False,  True])
A[mask]                          # rows 0 and 2, as a new (2, 2) array
mask.sum()                      # 2 (True counts as 1)
mask.mean()                    # 0.667: the FRACTION that is True
np.isin(labels, ['a', 'c'])     # membership test, one boolean per entry

```

Combine masks with `&`, `|`, `~`, always with parentheses: `(labels == 'a') | (labels == 'b')`. Writing `and` or `or` between arrays raises “truth value of an array is ambiguous”.

4.8 Sorting, unique values, and the upper triangle

```

v = np.array([0.3, 0.1, 0.7, 0.2])
np.argsort(v)                   # array([1, 3, 0, 2]): INDICES, smallest first
np.argsort(v)[::-1]             # largest first
np.unique(labels, return_counts=True) # (array(['a','b']), array([2, 1]))

iu = np.triu_indices(4, k=1)    # upper triangle of a 4 x 4, no diagonal
iu                                # (array([0,0,0,1,1,2]), array([1,2,3,2,3,3]))
M = np.arange(16).reshape(4, 4)
M[iu]                           # array([1, 2, 3, 6, 7, 11]): 6 = 4*3/2 entries

```

`argsort` returns positions, not values. `triu_indices(N, k=1)` is a *pair* of index arrays that walk the entries above the diagonal in a fixed order; `D[iu]` pulls those $N(N-1)/2$ entries out as a flat vector. If two matrices list the same stimuli in the same order, the same `iu` extracts corresponding pairs.

4.9 Reordering rows and columns together

```

order = np.array([2, 0, 3, 1]) # a permutation of all four indices
M[np.ix_(order, order)]        # rows AND columns in that order: the 4 x 4, rearranged
M[order, order]                 # NOT that: M[2,2], M[0,0], M[3,3], M[1,1] -> 4 entries

```

Two index arrays side by side are read as *pairs* of coordinates. `np.ix_` selects every combination of the specified rows and columns, which is what “sort the matrix by category” means. An order shorter than the matrix selects a sub-matrix.

4.10 Diagonals, percentiles, tolerances, and missing values

```
np.fill_diagonal(M, 0)           # in place; returns None
np.diag(M)                       # the diagonal as a vector
np.percentile(v, [2, 98])        # two numbers: 2nd and 98th percentile of v
np.allclose(a, b)                # usage pattern: equal within rtol=1e-5, atol=1e-8?
w = np.array([-1., 2., -3.])
np.where(w < 0, 0., w)           # array([0., 2., 0.]): a NEW array, entry by entry

B = np.array([[1., np.nan], [3., 4.], [5., 6.]])
np.isnan(B).sum()                # 1: how many NaNs
np.isnan(B).any(axis=0)          # array([False, True]): which COLUMNS have any
np.nanmean(B, axis=0)            # array([3., 5.]): mean ignoring NaNs
```

`np.allclose` answers “equal within the chosen absolute and relative tolerances”; its defaults accept differences much larger than floating-point roundoff, so pass `atol=` when the scale matters, and check shapes separately (arrays of different shapes may broadcast and still compare). Ordinary means, correlations and distances propagate NaNs; the nan-aware functions skip them, but an all-NaN column still has no mean. Replacing missing observations changes the analysis: use the preprocessing rule the notebook specifies.

4.11 Loading the data: .npz files

```
arc = np.load('activations.npz', allow_pickle=True) # usage pattern: dict-like
arc.files                                     # the keys, e.g. ['early', 'middle', 'late']
X = arc['late']                              # one array out
X.shape, X.dtype                             # e.g. (120, 4096) float32
X = X.astype(np.float64)                     # float32 -> float64 before computing distances
```

The notebooks load everything for you. Read the “Variables you will use” comments at the bottom of each setup cell; they list every name, shape and dtype. Cast to `float64` before distance calculations to reduce roundoff; some formulas can still leave residuals of order 10^{-8} where an exact zero is expected.

5 SciPy and scikit-learn: the calls the notebooks use

5.1 Distances: pdist and squareform

```
from scipy.spatial.distance import pdist, squareform
d = pdist(A, metric='euclidean') # CONDENSED: shape (n*(n-1)/2,), one per pair
d                                # array([4.472, 4.472, 2.828])
D = squareform(d)                # SQUARE: shape (3, 3), symmetric, zero diagonal
squareform(D)                    # and back to the condensed form
```

`metric` is any of 'euclidean', 'cosine', 'correlation', and others. When you supply precomputed dissimilarities, MDS expects the square form and linkage expects the con-

densed form. `squareform` checks that a square input is symmetric with a zero diagonal and raises if not; that is a useful check, not an obstacle, so find the cause before reaching for `checks=False`, which skips the check without repairing anything.

5.2 The scikit-learn pattern for embeddings

The embedding methods used here (PCA, MDS, t-SNE) follow one pattern: put the settings in the constructor, hand the data to `fit_transform`, and read anything it learned from an attribute with a *trailing underscore*.

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
Y = pca.fit_transform(A)          # A is (3, 2) in -> Y is (3, 2) out
pca.explained_variance_ratio_    # array([0.75, 0.25]): variance per component
```

`Y` holds one row of component scores per row of `A`. PCA centres each feature (column) across the rows; it does not centre each row across its features, and it does not rescale the features. The ratios are sorted, largest first, and sum to 1 when `n_components` equals the number of features.

5.3 MDS with a precomputed dissimilarity matrix

Multidimensional scaling takes a dissimilarity matrix and looks for points in the plane whose pairwise distances match it as well as possible. It starts from a random layout and moves the points to reduce the mismatch, so different starts can give different maps. The notebook provides the MDS call in Part 2; this is what its arguments mean.

```
from sklearn.manifold import MDS
mds = MDS(n_components=2, metric='precomputed', metric_mds=False,
          n_init=4, max_iter=400, random_state=0)  # usage pattern
Y = mds.fit_transform(D)      # D square (n, n) in -> Y (n, 2) out
mds.stress_                   # mismatch between map distances and disparities
```

- `metric='precomputed'`: `D` already holds dissimilarities; do not recompute them from `D` as if it were data.
- `metric_mds=False`: nonmetric MDS, which fits only the *rank order* of the dissimilarities. True fits their values.
- `random_state` fixes the random start so two runs agree; `n_init` runs several starts and keeps the best.
- `stress_` is comparable only between runs on the same input with the same stress definition. These argument names need scikit-learn ≥ 1.8 ; the Part 2 setup cell installs it. If the error in §8 appears anyway, restart the kernel and rerun the setup cell.

5.4 t-SNE

`sklearn.manifold.TSNE` follows the same constructor-then-`fit_transform` pattern and takes coordinates, shape `(n, d)`, not a distance matrix. The Part 4 notebook states the settings it wants; the arguments mean:

Argument	Meaning
n_components	dimension of the output; 2 gives an (n, 2) map
perplexity	roughly, how many neighbours each point tries to keep close; must be smaller than n
init	starting layout: 'pca' (deterministic) or 'random'
random_state	seed for the random parts; fix it to make a run repeatable
max_iter	how many optimisation steps to run

5.5 Correlations

```
from scipy.stats import spearmanr, pointbiserialr
res = spearmanr([1, 2, 3, 4], [1, 3, 2, 4])
res.statistic      # 0.8: the rank correlation rho
res.pvalue         # exists, but see below
pointbiserialr([0, 0, 1, 1], [1., 2., 5., 6.]).statistic # 0.970
np.corrcoef([1, 2, 3], [1, 2, 4])[0, 1]                # 0.982: Pearson r of the two inputs
```

Both inputs must be flat vectors of the same length. `np.corrcoef` returns the 2×2 matrix of correlations among its inputs (ones on the diagonal), so `[0, 1]` picks the entry for input 0 against input 1. To compare two symmetric RDMs over the same ordered stimuli, correlate their corresponding upper-triangle entries (§4.8). RDM entries share stimuli, so the usual p-value does not establish significance for an RDM correlation.

5.6 Fitting a curve: `curve_fit`, `polyfit`, `polyval`

Fitting and drawing are two separate steps. Fitting finds the parameter values that make a model function match the observed (x, y) pairs as closely as possible. Drawing evaluates the fitted function on a fresh, evenly spaced grid of x values (`np.linspace`) so the curve looks smooth; the grid is not the data.

```
from scipy.optimize import curve_fit

def line(t, m, c):          # first argument is the data; the rest are parameters
    return m * t + c

t = np.array([0., 1., 2., 3., 4.])
y = np.array([1., 3., 5., 7., 9.])
popt, pcov = curve_fit(line, t, y, p0=[1, 0])
popt          # array([2., 1.]): fitted m, c in the function's order
line(t, *popt) # the fitted values; * unpacks popt into m, c
np.linspace(0, 4, 50) # 50 evenly spaced points, for drawing a smooth curve
```

- `p0` is the starting guess. Choose values whose predicted curve has a plausible scale over the observed inputs; a poor start can send the fit to a wrong solution or no solution.
- `maxfev` limits the number of function evaluations for the default solver. If fitting fails, read the whole message and inspect the inputs, the model function and the starting values before raising it. Report a failure you cannot resolve; by itself it does not show that the model is wrong.

- `np.polyfit(x, y, deg)` fits a polynomial of degree `deg` and returns its coefficients, highest power first; `np.polyval(coef, x)` evaluates a polynomial with those coefficients at `x`.

R^2 , when a notebook asks for it, is provided as a function in that notebook. With $\hat{y}_i$ the model's prediction for observation i , it is $1 - \sum_i (y_i - \hat{y}_i)^2 / \sum_i (y_i - \bar{y})^2$: 1 is a perfect fit, 0 is no better than predicting the mean, negative is worse than that, and a constant target makes the denominator zero. To compare models directly, compute R^2 on the same observed response values; averaging or binning the data changes the target and its variance.

5.7 Hierarchical clustering

```
from scipy.cluster.hierarchy import linkage, fcluster, dendrogram
Z = linkage(pdist(A), method='average') # takes the CONDENSED form; Z is (n-1, 4)
```

Each row of `Z` records one merge: the two clusters joined, the height (distance) at which they joined, and the size of the result. `method` chooses how the distance between clusters is defined ('single': closest pair; 'average': mean over pairs). `fcluster(Z, t, criterion)` cuts the tree and returns one integer label per row: with `criterion='maxclust'` it forms at most `t` clusters (tied merge heights can produce fewer), with `criterion='distance'` it cuts at height `t`. `dendrogram(Z)` draws the tree. If you try both linkage methods, give the results different names rather than overwriting `Z`.

5.8 Silhouette scores

```
from sklearn.metrics import silhouette_samples
sil = silhouette_samples(Y, labels) # usage pattern: Y (n, d), one label per row
                                     # returns (n,): one score per point
```

For each point, the score compares its mean distance to its own group with its mean distance to the nearest other group. Near 1: much closer to its own group. Near 0: the two average distances are similar. Negative: closer to another group than to its own. It needs at least two groups and fewer groups than points. The function returns one score per image; the notebook says how to summarise those scores.

6 Sanity checks on a distance matrix

Run these every time you have made one, before anything downstream reads it. In a notebook, only the last bare expression in a cell is displayed, so print each check:

```
print('shape', D.shape) # (n, n)?
print('symmetric', np.allclose(D, D.T, atol=1e-8)) # 1e-16 off is roundoff
print('zero diagonal', np.allclose(np.diag(D), 0, atol=1e-8))
print('nonnegative', (D >= 0).all())
print('all finite', np.isfinite(D).all()) # catches nan AND inf
off = D[np.triu_indices(len(D), k=1)] # the off-diagonal entries, each pair once
print('range', off.min(), off.max())
```

If the diagonal is 10^{-8} rather than 0, inspect the values before changing them: a formula that reaches zero only up to roundoff is fine to clean with `np.fill_diagonal(D, 0)`, but decide

that after establishing the discrepancy is numerical, not before. If `np.abs(D - D.T).max()` is 10^{-17} , nothing is wrong. Cosine and correlation distances lie in $[0, 2]$: excursions of order 10^{-15} are roundoff; larger ones mean the formula or its inputs need checking; a zero-length or constant row needs the explicit policy the notebook states.

7 Plotting matrices and embeddings

```
fig, ax = plt.subplots(2, 3, figsize=(12, 7)) # a 2 x 3 grid; ax is a (2, 3) array
lo, hi = np.percentile(D, [2, 98])
im = ax[0, 1].imshow(D, vmin=lo, vmax=hi) # a matrix as an image
fig.colorbar(im, ax=ax[0, 1]) # its colour scale
ax[1, 0].scatter(Y[:, 0], Y[:, 1], s=8) # a 2-D embedding
ax[0, 0].set_title('...'); ax[0, 0].set_xticks([]); ax[0, 0].set_yticks([])
plt.tight_layout(); plt.show()
```

`vmin` and `vmax` set which values map to the two ends of the colour scale; values outside share the end colours but are not removed from the data. Use the limits the notebook specifies, label every panel, and show a colour bar, since panels with different limits cannot be compared by colour alone. To colour a scatter by category, loop over the categories and call `ax.scatter` once per group with a `label=`, then `ax.legend()`.

8 Common errors and what they mean

- `ValueError: operands could not be broadcast together with shapes (120,4096) (120,)`
A reduction dropped an axis. Decide which axis you meant and what shape the result should have; `keepdims=True` or `[:, None]` restores it (§4.6).
- `ValueError: shapes (120,4096) and (120,4096) not aligned`
Matrix product with inner dimensions that do not match. Work out the intended operation and its output shape before transposing anything: a `.T` on the wrong operand gives a valid but wrong matrix.
- `IndexError: index 120 is out of bounds for axis 0 with size 120`
Indices start at 0; the last row is 119.
- `ValueError: The truth value of an array with more than one element is ambiguous and/or between arrays, or if array:.` Use `&|` with parentheses for masks; use `.any()` or `.all()` for a single yes/no.
- `ValueError: Distance matrix 'X' must be symmetric`
`squareform` received a square matrix that is not symmetric. Find the asymmetry (`np.abs(D - D.T).max()`) and its cause before continuing.
- `ValueError: The condensed distance matrix must contain only finite values`
There is a `nan` or `inf` in the distances. Locate it (`np.argwhere(~np.isfinite(D))`) and trace it to its input.
- `TypeError: MDS.__init__() got an unexpected keyword argument 'metric_mds'`
scikit-learn older than 1.8. Restart the kernel and re-run the setup cell.

- `RuntimeError: Optimal parameters not found`
`curve_fit` did not converge. Check the model, the inputs and `p0`; then raise `maxfev`.
- `TypeError: 'numpy.float64' object is not callable`
A name that used to be a function is now a number, usually `r2 = r2(...)` or a variable named `sum`. Rename and re-run the definition.
- `NotImplementedError: 1b: write your code here, then delete this line.`
The guard line at the bottom of a TODO cell. Delete it once your code is in.
- **Unexpected nan results**, with no error at all. A nan in the input propagated, or a division by zero occurred. Check which axis the computation runs along: a row-wise correlation needs variation within each row; standardising units across images needs variation within each column.